Deploying decryption oracles for fun and non-profit: Backing up with friends and TEEs

Kanav Gupta
University of Maryland

Gabriel Kaptchuk*
University of Maryland

Ian Miers*
University of Maryland

Abstract

Secure backups are the Achilles' Heel of the E2EE ecosystem if they do not provide the same strong security properties as the E2EE messaging systems they support. They constitute a set of servers that, if compromised, would expose nearly all user messages. Unfortunately, state-of-the-art and deployed secure backup systems fail to consider forward secrecy and post-compromise security of these servers as first-order design constraints. Additionally, some proposals, in limited deployment, implicitly rely on the PKIs of trusted execution environments in order to provide security, creating a small number of keys whose compromise would be catastrophic.

We develop an elegant, efficient, and simple secure backup system that naturally addresses these issues by regularly rotating backup servers, each of which samples independent key material. To make this approach scalable, we design a *silent backup procedure*, reducing server load compared to state-of-the-art designs while providing improved security. Our design can be trivially extended to incorporate *social key recovery*, enabling more flexible deployment configurations. We carefully prove the security of our construction and benchmark it to show that it is deployment-ready. Our approach works on commodity hardware making it deployable without the resources needed for WhatsApp or Apple's Encrypted Backups.

1 Introduction

Modern end-to-end encrypted (E2EE) messaging systems are secure not just against eavesdroppers, but—to the maximum extent possible—attackers compromising key material. State-of-the-art systems, including Signal and WhatsApp, provide three crucial guarantees: (1) *forward secrecy*, ensuring that past, deleted messages remain secure even when an attacker compromises a client's device and keys; (2) *post-compromise security*, ensuring that temporary compromise does not expose future messages; and (3) no single point of compromise that would effect all users—no master key whose compromise would expose vast troves of communication.[1]

The strong security properties of encrypting messaging, however, break down, when we consider the infrastructure needed to provide backups, which have become an essential feature for real-world deployment and are. These servers, if compromised, would explore nearly all users' messages, making forward secrecy and post-compromise even more important for the backup infrastructure than it is for individual clients.

Deployed secure backup systems all share a single template: users encrypt their key material under a joint function of their password (or pin) and high-entropy key on the recovery server; typically, this involves the client deriving a key from their pin via an Oblivious Pseudorandom Function (OPRF) keyed by the server. In order to prevent dictionary attacks, the recovery server stores the high-entropy key within a secure compute environment (e.g., hardware security modules [15, 16, 40] or trusted execution environments [11]) that limits the number of times this function can be evaluated. To further strengthen the security properties of the system, the trust in the secure compute environment can be distributed in a threshold manner across multiple physical devices [12] or secure compute providers [11], ensuring backups remain secure even when an attacker compromises a subset of the recovery infrastructure.

Given that these backup system store key material for E2EE system, one might expect them to provide the same security properties in the event the keys they hold are compromised. Namely, forward secrecy would allow secure deletion of old backups and post-compromise security would ensure that a temporarily compromise of the server key material would not allow decryption of all future backups. On the contrary, however, achieving forward secrecy and post compromise security for these secure backup systems is an understudied problem and one given little practical consideration in system

*Joint senior authors. Order determined by a coin flip.

[1] PKI mapping usernames to keys, of course, comes close, but key fingerprints and key transparency make this difficult to exploit, especially at large scale.

designs—despite the fact that backups are a more concentrated target than client devices. Even worse, many designs introduce an oft-overlooked central point of failure: the PKI infrastructure used to identify a TEE is rooted by a master signing key held by the TEE manufacturer. We stress that these are not hypothetical or theoretical problems—failing to design robust *secure* backup systems immediately makes them the Achilles' Heel of the E2EE ecosystem.

Challenges of designing *robust* secure backup systems. Consider a baseline solution in which there is a single HSM or TEE storing the server's key material. Making such a system robust encounters the following challenges:

(1) *Failover and verifying hardware authenticity:* As in any cloud service, encrypted backup serves must handle failover by provisioning new servers. However, validating new TEE hardware relies on the hardware manufacturer's or cloud operator's trusted PKI, rooted in the a small number of master signing keys. Were these signing keys to be compromised, an attacker could impersonate genuine HSMs/TEEs, and gain access to all key material. This threat is not addressed in many proposals and, at present, the only known solution is to massively over provision capacity at initial setup, which is cost prohibitive for many organizations [15, 16].

(2) *Handling key rotation as we scale up:* In order to provide forward secrecy and post-compromise security, we must periodically rotate the server's key material. Because key rotation necessarily invalidates existing client backups, clients must generate a fresh backup. But, for the OPRF approach widely used in many backup schemes, this requires the client to interact with the service. Doubling the frequency of backups effectively doubles the load on the service, resulting in severe limits on the speed at which key material can reasonably be rotated and therefore extending the window of exposure for an enclave. This is in contrast to the fine grained forward secrecy and post compromise security achieved by E2EE messaging protocols.

(3) *Scaling out infrastructure and trust diversity:* Of course, we cannot run our backup system with only a single HSM/TEE for two reasons. First, like all services at scale, an encrypted backup service needs to scale out to run on many servers, since messaging services hundreds of million to billions of daily active users. Second, there is a vast literature demonstrating that HSMs' and TEEs' security guarantees are imperfect, especially in the face of highly-resourced and persistent adversaries [10, 34, 36–39]. As such, we must build a large cloud infrastructure and distribute our trust among a variety of hardware manufacturers and cloud providers. But, operating at scale makes verifying hardware authenticity and key rotation *significantly* more complex, expensive, and risky.

As an additional design constraint, we require systems that can be deployed for relatively low cost, allowing smaller and non-profit E2EE platforms (e.g., Signal) to provide strong security. This leads to the final design challenge:

(4) *Guess limits on commodity trusted hardware:* The security of a pin-based encrypted backup system depends almost entirely on the systems ability to limit guess attempts for the low entropy pin. Here, its critical that the system is secure against *rollback attacks* that reset the guest counter. One option is to use hardware security modules with strong rollback preventions. Indeed this is the approach taken by Apple and WhatsApp [15, 40] These, however, incur some substantial cost.

Another option is to use commodity Trusted Execution Environments. However, widely available and concretely affordable commodity TEEs, including SGX on server chipsets, TDX, AMD SEV-SNP, and AWS Nitro (see [3] for a detailed discussion), either do not provide protected counters or provide them with far lower security. The standard solution to resolve this is to operate a consensus protocol over the TEEs, reducing the security of the protocol to the security of the consensus protocol. This is a security reduction, as semi-synchronous consensus protocols are only robust against adversaries that corrupt less than 1/3 of the nodes (essentially limiting client's abilities to pick their own trust diversity) and consensus critically relies on carefully controlling the number of adversarial nodes that can participate in consensus. Alternatively, the system could leverage an external outsource consensus to a proof-of-stake cryptocurrency like Ethereum, but a high-resource attacker could simply focus on compromising the keys that hold enough *stake* (or even temporarily buy a large amount of stake) to locally fork the protocol. No matter the specific consensus approach taken, it will also drastically increases the trusted compute base of the TEE, since it now needs to handle consensus.[2]

Simultaneously solving all these problems efficiently is *necessary* before secure backup systems can safely be deployed. We emphasize that the choice of guess limiting mechanism to solve (4) is completely orthogonal to the solutions used to solve challenges (1), (2), and (3). Specifically, the design space for (1), (2), and (3) assumes the existence of *some* guess limiting approach, and a service provider can choose a solution to (4) that fits the security and efficiency profile they need. Looking ahead, we develop a protocol for (1), (2), and (3) and then explore combining it with multiple guess limiting approaches, ranging from the standard approach with HSMs

[2]For a detailed review of rollback prevention techniques in TEEs, please refer to Section 8.

to an alternative using a proof of work blockchain. We stress that the approach in this paper is not dependent on using a blockchain for (4).

1.1 Our Contributions

In this work, we design an efficient, dynamically scalable secure backup system that carefully takes deployment constraints into consideration. Our work leverages three observations: (1) if we can design a non-interactive backup procedure, clients can independently decide how to distribute trust across a heterogeneous backup infrastructure; (2) letting each TEE sample key material independently reduces the need for enclaves to mutually authenticate. Additionally, with a non-interactive backup procedure, the client doesn't need to interact with each TEE in order to use them as a backup resource; and (3) we can provide forward secrecy and post-compromise security naturally by allowing the set of active backup servers evolve and churn over time. Again, a non-interactive backup procedure makes this strategy possible, as frequently refreshing backups does not incur any server load. Our system has the following desirable properties:

- *Extraordinarily simple design:* Our system's core design is a careful composition of a set of well-studied cryptographic primitives for which there are existing efficient, robust implementations. We believe that this simplicity is a good sign that our approach is the "right way" to think about this problem.

- *Improved security guarantees:* We are able to significantly improve the security guarantees offered compared to state-of-the-art. Our construction provides natural forward secrecy and post-compromise security while actually *decreasing* the interaction required between clients and servers. Additionally, we provide some security in the event of catastrophic compromises: if the TEE PKI is compromised, the attacker doesn't get access to the key material stored in existing TEEs, even if trust in them is rooted in the compromised PKI. Using a compromised PKI to access messages requires announcing new (inauthentic) TEEs and waiting for *clients* to migrate their backups—a potentially slow and detectable process.

- *Configurable trust diversity and social key recovery:* Our system is highly configurable, allowing clients to ad-hoc customize their backup configurations, e.g., distribute trust across different TEEs, regions, cloud provider, and architectures or choose their preferred balance between security and availability. In fact, our system architecture naturally allows interoperability with *social key recovery*, in which some amount of trust is placed in a users personal contact, rather than exclusively rooting trust in cloud platforms. This is possible because interaction is only required during recovery and TEEs need not be mutually aware.

- *Rigorous proof of security:* We provide a simulation-based security definition for our system and rigorously prove that our protocol realizes this definition.

2 Technical Overview

Our work considers a client holding a secret m.[3] Our construction is agnostic to the specific utility of m, but we encourage the reader to imagine m is a *key encryption key* which encrypts some *data encryption key material* used to encrypt a users' message history. The client wants to back up m under a low-entropy password[4] pwd in the presence of multiple recovery servers $S_1, S_2, \ldots, S_n$, each running a TEE.[5] These TEEs might be of different types (i.e., TDX, SGX, or Nitro), in different cloud environments (i.e., Azure, Google Cloud, or AWS), or in different jurisdictions. Importantly, we assume that each of these servers samples their key material completely independently and publishes their public key, along with an attestation that they are running the correct software, publicly. The client's software starts by building an access policy $\mathbb{A}$ over these servers (e.g., 7 of 10 TDX nodes on Azure, 5 of 9 AMD SEV-SNP nodes on GCP, and 8 of 11 Nitro nodes on AWS) and then generating secret shares $m_1, m_2, \ldots, m_n \leftarrow \mathsf{Share}_{\mathbb{A}}(m)$ such that m can only be reconstructed using shares that satisfy $\mathbb{A}$. For simplicity, one can think of $\mathbb{A}$ as a threshold access structure and the secret sharing scheme as Shamir secret shares [35], but other access structures (like threshold of thresholds) are also supported.

In existing approaches [11, 40], the client would then specify the password pwd and interact with the TEE within each server S_i to encrypt m_i as

$$s_i = m_i + \mathsf{OPRF}_{k_i}(\mathsf{pwd}),$$

where OPRF is an oblivious pseudorandom function keyed with a high-entropy server key k_i stored in the TEE. Our work takes a slightly different approach: the client samples an OPRF key k_i *locally* for each server S_i and then locally evaluates the OPRF and computes s_i as before. Additionally, the client then encrypts the key pair under S_i's public encryption key pk_i as

$$ct_i \leftarrow \mathsf{Encrypt}_{\mathsf{pk}_i}(k_i).$$

The client's backup then constitutes a set of pairs $(ct_1, s_1), (ct_2, s_2), \ldots$ all of which can be stored in arbitrary cloud storage (e.g., storage provided by the E2EE service provider or Google Drive). To recover m, the client can download the (ct_i, s_i) tuples, send ct_i to S_i's TEE, and then query using the oblivious evaluation protocol on pwd, unmasking

[3] To reduce ambiguity, we refer to the client's data as a message, even though it will often, in practice, be cryptographic key material.

[4] Throughout the rest of this work, we refer to the client's low-entropy secret as a *password*, although in practice it may also be a pin.

[5] For convenience, we use the term server and TEE interchangeably, as the "important" part of the servers we consider are their TEEs.

the share m_i. Finally, once enough m_i have been recovered, the client can reconstruct m.

Notice that this backup procedure is fully *silent*, in that there is no interaction between the client and S_i. While this will improve the *efficiency* of the backup system, this alone is not sufficient to provide improved security.

Improving security via key rotation. Rather than thinking of the set of servers $S_1, S_2, \ldots, S_n$ as *static*, we accept that the set will change, allowing for hardware failures, reboots, cloud migration, or dynamic scaling. Whenever a server is removed from the active set—even if it is just for a temporary reboot—the key material $(\mathsf{pk}_i, \mathsf{sk}_i)$ it stores will be permanently deleted and each new—or rebooted—server will sample a fresh key pair. Rather than seeing this as a "bug," we leverage this churn into a free source of gradual key rotation. Namely, we can require all servers to reboot regularly, even if it is not necessary for logistical reasons, in order to provide stronger security guarantees.

When a client wants to back up their message m, they use (a subset of) the currently active servers and run the backup procedure outlined above. Eventually, a sufficiently large number of their chosen servers may go offline—securely deleting the TEE encryption keys in the process—such that it would be *impossible* to reconstruct m, even with access to the correct pwd. At the same time, a set of new servers with fresh key material will come online. This churn means the client must *refresh* their backup of m, generating a new set of pairs $(ct_1, s_1), (ct_2, s_2), \ldots$ with respect to the new active set of servers. Alternatively, the client could sample a new m' to replace m entirely and back m' up instead.[6] Importantly, the client should not wait until their old backups are non-recoverable, but instead should regularly be refreshing their backups such that they have the necessary redundancy. Importantly, clients can't simply just "migrate" a masked share m_i to another server, or they cannot possibly achieve forward secrecy. Instead, the client should generate a *fresh* secret sharing of m each time they renew, allowing their old backups to completely expire.

Notice that this approach immediately provides forward secrecy and post-compromise security. We get forward secrecy because old backups eventually become fully non-recoverable, even if all key information in the system is later compromised. Similarly, we get post-compromise security because each new server samples their encryption key independently, allowing the security of the system to "heal" over time.

Moreover, we observe that the TEE PKI is no longer a single point of failure in our system, despite the fact that we make use of cheap cloud TEEs. If an adversary were to compromise the TEE PKI, they could convincingly impersonate authentic TEEs and advertise valid-looking encryption public keys for backup, but there is no synchronization *between* TEEs—even those that are backed by the same TEE PKI. Over time, some clients may migrate their backups (at least partially) to these malicious servers. Importantly, however, this process would likely be somewhat slow, as it relies on client-driven actions. Moreover, the malicious servers would need to announce themselves publicly in order to successfully exfiltrate user messages; this provides an opportunity for the service provider to identify a failure at halt the system's operation. Given the risk of TEE PKI compromise, it may make sense for clients to slowly ramp up their trust in newer TEEs, either by designing an access policy $\mathbb{A}$ that puts little trust in them or by migrating shares with probability proportional to their uptime. Paranoid clients or applications using the backup service, can effectively be configured in a fail-safe model, where excessive churn in TEEs, e.g., by an adversary spinning up many sybil servers, would not be selected, simply halting future backups while previous ones slowly expire.

We note that our approach is only feasible because we have a non-interactive backup process. If client-server interaction were required in order to refresh backups, the TEEs would become saturated by the hundreds of millions of users who would need to perform such an action. Instead, interactions between clients and server are only necessary during *recovery*, which should be relatively less frequent and independent of how aggressive clients are in achieving fine grained forward secrecy and post compromise security for backups.

Synchronizing query counts. So far, we have implicitly assumed that there was some mechanism to rate-limit the queries sent to each server. Because the cloud-provisioned TEEs we use have no non-malleable state, we must leverage a mechanism outside of the TEE to enforce limits. Existing work's decision to rely on consensus among the TEEs [11] makes the TEE PKI a vulnerability again, as an attacker might be able to subvert the consensus process if they controlled enough TEEs, allowing them to launch efficient *online* dictionary attacks, which may be enough to quickly brute-force the low entropy pins used in some configurations.

In our work, we turn to existing consensus networks as an alternative tool to root consensus, conceptually building on the approach of Kaptchuk, Green and Miers [23]. Specifically, we propose that the recovery servers rely on the extraordinarily high cost of forking a proof-of-work blockchain (e.g., Bitcoin [33] or Zcash [5, 31]) to prevent an attacker from circumventing the rate limiting process.[7] The most naive way to do this is have clients generate a chain of transactions, in which each transaction embeds a specific password attempt. Because transactions can only be spent once, this allows TEEs

[6] When m is a key encryption key, this would require re-encrypting just the data encryption key rather than the full backup (e.g., message history). Looking ahead, the data encryption key material may also change over time to facilitate secure deletion of messages in the backup history or providing post-compromise security.

[7] It is important that the blockchain use proof-of-work rather than proof-of-stake, as we want to avoid the existence of key material that could be stolen in order to allow brute-force attacks. Moreover, highly-resourced attackers could also stake a large amount of money temporarily in order to launch such an attack before withdrawing their fund, making this attack very inexpensive.

to robustly count the number of previous password attempts corresponding to a particular backup and limit the number of query to which they will respond (e.g., ten).

An important, and often overlooked, part of guess counting is preventing the attacker from copying a existing backup directly into a new backup with a fresh guess counter, thereby bypassing the guess limit. In our work, we do this by assigning each backup with a randomly sampled backup identifier BID. Each ciphertext then embeds BID as $ct_i \leftarrow \mathsf{Encrypt}_{\mathsf{pk}_i}(k_i, \mathsf{BID}, \mathsf{lim})$. When logging a new guess attempt, it is associated with the identifier BID and the enclave verifies that this BID matches the one in the ciphertext. Critically, this approach *requires* a non-malleable public key encryption scheme, such that an attacker cannot blindly modify the BID stored in a ciphertext.

Alterative approach for synchronizing query counts and design trade offs. Our proposed approach for query synchronization—using a proof-of-work blockchain— incurs delays: transactions must be confirmed before a client can complete their recovery, requiring 10 to 60 minutes for Bitcoin and 2 minutes to 15 minutes for Zcash, depending on number of confirmations required. While these delays are not insignificant, we note that recovery should be a relatively rare event for any given user and rarely is instantaneous recovery necessary—especially if it comes at the cost of reduced security. Finally, we note that specific consensus mechanism is completely orthogonal to the rest of our design; designers could use consensus between the TEEs or some small number of HSM counters, choosing to sacrifice some security for reduced latency, or combine all approaches for defense in depth.

2.1 Extensions

Social key recovery. While all existing, deployed secure backup systems only use service-provider provisioned resources as recovery servers, a popular idea in the literature is *social key recovery*, in which clients put some trust into their contacts instead [9, 25, 28, 32]. While promising, this approach has proved difficult to operationalize.

Having a non-interactive backup procedure, however, makes social key recovery trivial: clients can simply encrypt some of the shares k_i to their contacts. When the user's contact's devices have secure elements, the construction can remain exactly the same, although it may be prudent to require a users' contacts to manually approve guess attempts or restrict guesses to be face-to-face (i.e., over NFC). When these devices have no secure element, there is no way to prevent a client from locally running a dictionary attack to attempt to recover m_i. To mitigate the risk of such an attack, we simply pick a secret sharing scheme with shares that are computationally close to uniform when below the reconstruction threshold (e.g., Shamir secret shares), which prevents an attacker from checking if their password attempts are correct.

On-demand forward security via. puncturable public key encryption. The above design expires existing backups *slowly*, as a user must either wait for enough servers to churn or must exceed their maximum number of guesses. Letting clients request that a TEE delete their key material would immediately expire their existing backup, this the TEE's key material is reused across all clients, making this approach a dead-end.

If deployments want to enable proactive deletion, they can replace the public key encryption scheme in the above sketch with a *puncturable public key encryption scheme* [17]. In this scheme, the TEEs can locally modify their secret key material such that they can no longer decrypt specific ciphertexts, e.g., either from specific users or encrypted under some known tag. Using such a scheme would allow clients to request recover servers to puncture out their existing backups without impacting the recoverability of other clients. We note that this approach needs to be treated carefully, as many TEEs are vulnerable to rollback attacks, which would potentially allow an adversary to "un-puncture" a secret key.

Efficiently deleting backed up files. Until this point, we have largely elided how m should would be used within an end-to-end application. Recall that m could be a key encryption key which is used in a KEM/DEM paradigm (i.e., encrypting a data encryption key material, which is actually responsible for encrypting the user's message history). By providing forward secrecy to m, the data encryption key can also be rotated, thereby securely deleting old versions of backups. In principle, this makes it possible to delete specific messages with a users' message backups. A naive approach to this would require the user to sample a new data encryption key d, re-encrypt their entire message history besides the message that will be deleted under d, and then sample a fresh m' to encrypt d. Finally, m' can be backed up. Recently Dodis, Jost, and Marcedone describe Compact Key Storage [13, 14], an elegant technique for reducing this linear overhead to logarithmic while keeping $|d|$ logarithmic in the message history . Their approach can be directly integrated with our own to facilitate efficient deletion of files.

2.2 Threat Model

In our work we assume a powerful state-sponsored attacker who is interested in (1) mass-compromise of many users backups in order to conduct mass surveillance, and (2) the targeted compromise of high-value targets' backups.[8] We begin with the new threats we consider in this work:

– *Compromise TEE PKI:* All modern TEEs embed some attestation or measurement protocol, which allows clients to

[8] Of course, individuals who are very high-value, e.g., leaders of opposition movements or diplomats, should likely not be using backups at all, as they are a usability feature that incurs security risk; this approach has been embraced by Apple's Lockdown Mode, which trades worse functionality for much stronger security guarantees.

verify that (1) the endpoint with which they are interacting is an authentic TEE, and (2) the software the TEE is running matches some pre-determined hash. The security of this mechanism is rooted in key material held by the TEE manufacturer. In a significant departure from prior work, we assume that the adversary can compromise this high-value key material.

– *Slowly compromise TEEs:* Clearly there can be no meaningful security if the adversary can arbitrarily compromise all TEEs in the system, as dictionary attacks will immediately be possible. Instead, we aim for the next, weakest security assumption: we assume that the adversary can slowly compromise TEEs and TEE types, such that over a long time all TEEs will eventually be compromised, but not at the same time.

Additionally, the adversaries we consider can perform the following, standard actions:

– *Compromise service provider:* We assume the service provide itself can be compromised, either because they are legally compelling to take some particular action, an internal malicious actor, or a system breach. As a result, any key material held by the service provide that is not stored in a TEE can be accessed by the adversary and all server-side data can be arbitrary modified.

– *Compromise cloud provider:* The adversary can direct the cloud hosts on which TEEs are executed to act arbitrarily. In addition to being able to read or modify all messages exchanged between clients and TEEs, the adversary can launch *rollback* attacks (either using software snapshots or potentially even physical access to the device), in which the attacker attempts to revert the state of the TEE to some previous, authentic state.

– *Act as a malicious client:* The adversary may also act as a malicious client interacting with the system, potentially reusing or modifying messages sent by other, honest clients.

– *Post to public ledger:* Our solution relies upon the unforgability of a public ledger as a synchronization primitive. Thus, we consider adversaries that cannot forge proofs of inclusion on a known public ledger (e.g., locally generate forks of the Bitcoin blockchain). The adversary may, however, post arbitrary messages onto the ledger.

We enumerate our desired security properties, both informally and formally, in Section 4.

3 Preliminaries

Notation For integers $a \leq b$, we use $[a,b]$ to denote the set $\{a,\ldots,b\}$. For integer n, $[n]$ denotes $[1,n]$. For a set S, we use $\mathcal{P}(S)$ to denote its power set. For a deterministic algorithm A, we use $y = A(x)$ to denote running A on input x to get y. In case of randomized algorithm, we use '$\leftarrow$' instead of '$=$'.

Secret Sharing. Our construction depends on a (generalization of) threshold secret sharing scheme, where the distribution of any subset of shares below threshold are uniform.

Definition 1. *For an integer n and a monotonic access structure $\mathbb{A} : \mathcal{P}([n]) \to \{0,1\}$, a $(\mathbb{A},n)$ secret sharing scheme is a set of algorithms* $(\mathsf{Share}, \mathsf{Reconstruct})$ *with the following syntax:*

1. $\mathsf{Share} : I \to \mathcal{S}^n$*, a randomized algorithm that splits a message (from domain I) into n shares (in domain $\mathcal{S}$)*

2. $\mathsf{Reconstruct} : \mathcal{S}^{\leq n} \to I \cap \perp$*, a reconstructing algorithm that combines shares output by* Share.

We require our secret sharing scheme to satisfy the following two properties:

1. Correctness: *For all $x \in I$, if $(x_1, x_2, \ldots x_n) \leftarrow \mathsf{Share}(x)$, then for all sets $P \subseteq [n]$ such that $\mathbb{A}(P) = 1$,* $\mathsf{Reconstruct}(\{x_i\}_{i \in S}) = x$.

2. Uniformity: *For all $x \in I$, if $(x_1, x_2, \ldots x_n) \leftarrow \mathsf{Share}(x)$, then for all sets $P \subseteq [n]$ such that $\mathbb{A}(P) = 0$, $\{x_i\}_{i \in P}$ is computationally indistinguishable from uniform distribution over $\mathcal{S}^{|P|}$*

Public Key Encryption. We revisit the definition of CCA-secure public key encryption:

Definition 2 (CCA Public Key Encryption)**.** *A CCA Public Key Encryption scheme is a set of algorithms* $(\mathsf{KeyGen}, \mathsf{Encrypt}, \mathsf{Decrypt})$ *that satisfies two properties:*

1. Correctness: *For all messages m, and* $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$, $\mathsf{Decrypt}(\mathsf{sk}, \mathsf{Encrypt}(\mathsf{pk}, m)) = m$.

2. CCA-Security: *The advantage of all adversaries $\mathcal{A}$ in the following game with challenger $\mathcal{C}$ is negligible:*

   (a) *$\mathcal{C}$ runs* $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$ *and sends* pk *to $\mathcal{A}$*

   (b) *$\mathcal{A}$ picks two messages m_0, m_1 of same length and sends it to the challenger.*

   (c) *$\mathcal{C}$ picks a random bit b and sends $c \leftarrow \mathsf{Encrypt}(\mathsf{pk}, m_b)$ to $\mathcal{A}$.*

   (d) *$\mathcal{A}$ can request polynomial number of decryption queries from the challenger, as long as the query is not c.*

   (e) *$\mathcal{A}$ outputs b' and wins if $b = b'$.*

2HashDH OPRF. Jarecki et al [20] provide a construction for a minimally interactive UC-secure oblivious PRF (OPRF). Informally, an OPRF allows a client, with input x, to obliviously learn $F(k,x)$ for a target PRF F, where the PRF key k is held by a server. For a formal definition, we refer the reader to [20].

$\mathcal{F}_{\mathsf{aoKVS}}$

Setup: Initializes an empty public list of lists L.

On input $(\mathsf{MakePost}, \mathsf{BID}, \mathsf{post})$ from P:

1. Append post to the end of $L[\mathsf{BID}]$ (create a list if it doesn't exist).
2. Send $(\mathsf{MakePost}, \mathsf{BID}, \mathsf{post})$ to $\mathcal{S}$ (if $P \neq \mathcal{S}$)

On input $(\mathsf{GetPost}, \mathsf{BID})$ from P:

1. Set $T \leftarrow L[\mathsf{BID}]$.
2. Set $\ell \leftarrow |T|$ and $\mathsf{post} = T[\ell - 1]$
3. Send back (post, ℓ) to P.

Figure 1: Append-only Key Value Store Functionality, $\mathcal{F}_{\mathsf{aoKVS}}$.

Jarecki et al's 2HashDH construction computes a target PRF $F(k,x) = H_2(x, H_1(x)^k)$, where H_1 and H_2 are random oracles. $H_1 : \{0,1\}^* \to \mathbb{G}$ converts a low entropy x into a group element, and $H_2 : \{0,1\}^* \times \mathbb{G} \to \mathbb{D}$ converts x and group element $\mathbb{G}$ into the target output domain $\mathbb{D}$. In our construction, $\mathbb{D}$ is going to be the share space of the corresponding secret sharing scheme.

To compute this PRF obliviously, the client first sends $u = H_1(x)^r$ for a random scalar r, to the server. The server, sends back $v = u^k = H_1(x)^{rk}$ to the client. The client unblinds v and hashes the output along with x to get the desired output; that is, it computes $H_2(x, v^{1/r}) = H_2(x, H_1(x)^k)$.

Append-only Key Value Store. Our construction makes use of an append-only key value store functionality $\mathcal{F}_{\mathsf{aoKVS}}$, as described in Figure 1. The functionality is reminiscent of a transparency log; however, we additionally require the functionality to return the number of values under the requested key. There are many ways in which such a functionality could be instantiated; most key-transparency systems [8, 18, 26, 27, 30] could be trivially reconfigured to serve as such a system. In our implementation, we take a more direct approach (building on the ideas in [23]) by simply committing to a plaintext dictionary and embedding this commitment into a chain of Bitcoin transactions. See Section 6 for details.

4 Definition: Password-Protected Distributed Key Backup and Recovery

In this section, we give both informal and formal security notions for our system. We note that the existing literature on secure backups tends to rely on informally security arguments, which can lead to missing subtleties. Thus, we choose to provide both.

4.1 Informal Security Goals

We begin by enumerating a set of informal security goals:

(1) *Client configurable trust:* Clients should have control over their own risk tolerance by arbitrarily configuring a threshold access policy for their backups among the set of online servers.[9] As long as the set of compromised TEEs does not satisfy that access policy, then the adversary can make no more than some public known limit lim number of password guesses. Beyond these guesses, the adversary can learn nothing about the contents of the backup.

(2) *Forward secrecy:* A backup system should allow clients to delete subsets of their data. As such, old backups must either naturally expire or expire in response to user requests. This should hold against an adversary which compromises *all key material in the system* after expiry.

(3) *Post-compromise Security:* In the event that the adversary does successfully compromise enough servers to access a clients' backups, we require that the system automatically "heal." Specifically, messages that are backed up *after* the adversary is no longer corrupting the system cannot be decrypted using the key material the adversary has already accessed.

(4) *Existing backups remain confidential under TEE PKI compromise:* In the event that the attestation keys for the TEEs are compromised, we require that the confidentiality of existing backups are not compromised. It is clear that this compromise will eventually result in the system failure, but this failure should be *slow* and *highly noticeable*, such that administrators can halt the system's use.

(5) *Security against slow-moving adversaries:* Confidentiality of a client's backup should be preserved against a slow-moving adversary that eventually could corrupt all the recovery servers. Such an adversary may not be able to *simultaneously* corrupt all the servers, but could, e.g., over years, gather all long-term servers' key material.

(6) *Security under password reuse:* Clients will inevitably reuse passwords over time. As such, we require that the recovery procedure does not reveal anything about the password.

[9] Our system is technically slightly more flexible: any access policy is possible, providing there exists a secret sharing scheme for that access policy such that any set of shares that do not satisfy the policy appear pseudorandom. This is trivially satisfied by Shamir secret sharing [35] as well as the threshold-of-thresholds generalization.

$\mathcal{F}_{\mathsf{KeyBackup}}$

Setup: Initiate $\mathsf{Active}, \mathsf{Corr}, \mathsf{Unregistered}, \mathsf{Backups}$ and $\mathsf{RecoveryAttempts}$ as empty lists.

Server Management: We assume Sim is aware of all interactions in this section.

On input $(\mathsf{Register}, S, \mathtt{t})$ from S (or Sim if S is corrupt):

1. If S is already in Active or $\mathsf{Unregistered}$, abort.
2. Append S to Active
3. If S is corrupt, append S to Corr.
4. After time $\mathtt{t}$ has passed and $S \notin \mathsf{Corr}$ at that point in time, invoke $(\mathsf{Unregister}, S)$.

On input $(\mathsf{Corrupt}, S)$ from Sim:

1. If S is already in Corr, abort.
2. Append S to Corr.

On input $(\mathsf{Unregister}, S)$ from S (or Sim if S is corrupt):

1. Remove S from Active and append S to $\mathsf{Unregistered}$

Backup Phase

On input $(\mathsf{Backup}, \mathsf{BID}, \mathbb{S}, \mathbb{A}, \mathsf{pwd}, \mathsf{lim}, m)$ from C (or Sim if C is corrupt):

1. If C is not corrupt, sample a unique BID and use it instead. `//Honest clients use high-entropy` BID
2. If C is corrupt and there already exists a record associated with BID in $\mathsf{Backups}$, sample a unique corrupt identifier backup ID CBID and record $(\mathsf{CBID}, \mathsf{BID}, \mathbb{S}, \mathbb{A}, \mathsf{pwd}, m, \mathsf{lim})$ to $\mathsf{Backups}$. Otherwise, record $(\perp, \mathsf{BID}, \mathbb{S}, \mathbb{A}, \mathsf{pwd}, m, \mathsf{lim})$ to $\mathsf{Backups}$.
3. Send $(\mathsf{Backup}, \mathsf{BID}, \mathsf{CBID}, \mathbb{S}, \mathbb{A}, \mathsf{lim})$ to Sim.
4. Set $\mathsf{ctr}[\mathsf{BID}] = 0$
5. Send $(\mathsf{CBID}, \mathsf{BID})$ to C.

Recovery Phase

On input $(\mathsf{InitRecovery}, \mathsf{CBID}, \mathsf{BID}, \mathsf{pwd}')$ from C (or Sim if C is corrupt): `//Honest clients set` $\mathsf{CBID} = \perp$

1. Retrieve record $(\mathsf{CBID}, \mathsf{BID}, \mathbb{S}, \mathbb{A}, \mathsf{pwd}, m, \mathsf{lim})$ from $\mathsf{Backups}$, and abort if it doesn't exist.
2. Set $\mathsf{attempt} = \mathsf{ctr}[\mathsf{BID}]$
3. Record $(\mathsf{CBID}, \mathsf{BID}, \mathsf{attempt}, \mathbb{S}, \mathbb{A}, \mathsf{pwd}, \mathsf{pwd}', m, C, \mathsf{lim})$ to $\mathsf{RecoveryAttempts}$
4. Send $(\mathsf{RecoveryStart}, \mathsf{CBID}, \mathsf{BID}, \mathsf{attempt})$ to Sim and all $S \in \mathbb{S}$.
5. Update $\mathsf{ctr}[\mathsf{BID}] = \mathsf{ctr}[\mathsf{BID}] + 1$

On input $(\mathsf{AllowRecovery}, S, \mathsf{CBID}, \mathsf{BID}, \mathsf{attempt})$ from S (or Sim if S is corrupted):

1. Fetch record $(\mathsf{CBID}, \mathsf{BID}, \mathsf{attempt}, \mathbb{S}, \mathbb{A}, \mathsf{pwd}, \mathsf{pwd}', m, C, \mathsf{lim})$ from $\mathsf{RecoveryAttempts}$, and abort if it doesn't exist
2. Check if $S \in \mathbb{S}$ and $S \in \mathsf{Active}$.
3. Mark S as ready for recovery for BID.

On input $(\mathsf{FinalizeRecovery}, \mathsf{CBID}, \mathsf{BID}, \mathsf{attempt})$ from Sim:

1. Fetch record $(\mathsf{CBID}, \mathsf{BID}, \mathsf{attempt}, \mathbb{S}, \mathbb{A}, \mathsf{pwd}, \mathsf{pwd}', m, C, \mathsf{lim})$ from $\mathsf{RecoveryAttempts}$
2. Let $\mathbb{Q} \subseteq \mathbb{S}$ be ready servers for BID. Check if $\mathbb{A}(\mathbb{Q}) = 1$.
3. Check if $\mathsf{ctr}[\mathsf{BID}] \leq \mathsf{lim}$, send abort if not.
4. if $\mathsf{pwd} = \mathsf{pwd}'$ send $(\mathsf{Success}, m)$ to C, otherwise send $(\mathsf{Failure}, 0^\lambda)$.

Adversary Endpoints

On input $(\mathsf{LeakKey}, \mathsf{CBID}, \mathsf{BID})$ from Sim:

1. Retrieve record $(\mathsf{CBID}, \mathsf{BID}, \mathbb{S}, \mathbb{A}, \mathsf{pwd}, m, \mathsf{lim})$ from $\mathsf{Backups}$ and send abort if it doesn't exist
2. If $\mathbb{A}(\mathbb{S} \cap \mathsf{Corr}) = 1$, send (m, pwd) to Sim, otherwise send abort

Figure 2: Password-protected Key Backup Functionality, $\mathcal{F}_{\mathsf{KeyBackup}}$

(7) *No offline dictionary attacks:* Just as in OPAQUE [22], we require that the system prevents offline dictionary attacks. Specifically, any computation the adversary does *before* compromising enough TEEs to satisfy the access policy should be useless.

4.2 Formal Security Definition

We model the ideal functionality of Password-Protected Distributed Key Backup and Recovery in $\mathcal{F}_{\mathsf{KeyBackup}}$ in Figure 2.

$\mathcal{F}_{\mathsf{KeyBackup}}$ provides four kinds of endpoints: (1) server management endpoints, (2) backup phase endpoints, (3) recovery phase endpoints, and (4) adversary specific endpoints. Throughout the ideal functionality, we make no differentiation between servers and the TEEs they serve, as the only logcially important component is the TEE.

The *server management* endpoints are used to express the ways in which servers are added and removed during the normal course of running the system. Rather than pre-specifying the churn pattern, we simply allow the environment drive this process (and, thus, we should think of it as adversarially controlled). This allows our definition to be *general* over all server configuration patterns. There are three endpoints: (1) Register, which registers a new server and sets S into an *active* state. Servers are asked to set an up-time t, which indicates the *maximum* amount of time the server will remain online; (2) Corrupt, which corrupts a server; and (3) Unregister, which takes a server offline. Note that once a server goes offline via Unregister, it can never return to an active state.

The *Backup phase* is simple: the client simply chooses a backup identifier BID, a set of servers $\mathbb{S}$, an access structure $\mathbb{A}$, a password pwd, a attempt limit lim and the secret material they want to backup K and sends them to the ideal functionality via the Backup call. $\mathcal{F}_{\mathsf{KeyBackup}}$ stores this data internally and sets the password guess counter for this backup to 0. The ideal functionality enforces that honest clients always use different (with all but negligible probability) BID, but allows malicious clients to adversarially choose their BID, potentially overlapping with a honest client's BID. In this case, we sample a corrupt backup identifier CBID to help the ideal functionality disambiguate between the two. In this case, all backups that share the same BID will share the same guess counter whenever someone starts the recovery process.

Recovery phase is a multi-step process, and hence, we have three endpoints for a single recovery. The first endpoint, InitRecovery, is for the client to initiate a recovery attempt on BID using password pwd'. $\mathcal{F}_{\mathsf{KeyBackup}}$ contacts all of the servers in $\mathbb{S}$ for their approval and increments the attempt counter. Then, it waits for the approvals of server, through the endpoint AllowRecovery, which the servers use to submit their approval. Then, in FinalizeRecovery, $\mathcal{F}_{\mathsf{KeyBackup}}$ checks if enough servers have responded, if the attempt limit is not crossed, and if the password is correct. If all checks pass, it returns the backed up key m. Note that the FinalizeRecovery endpoint can be only initiated by the adversary, as in the real world, a corrupt service provider can always deny servicing a recovery request.

Finally, our *adversary specific endpoint* LeakKey logically lets the adversary exfiltrate a secret m from the backup infrastructure if and only if it has corrupted enough servers to satisfy the access structure $\mathbb{A}$.

4.3 Justifying $\mathcal{F}_{\mathsf{KeyBackup}}$ Meets Our Goals

Our ideal functionality does not explicitly speak to our main security goals, but rather provides them implicitly. For clarity, we now explicitly argue why $\mathcal{F}_{\mathsf{KeyBackup}}$ provides forward secrecy, post-compromise security, and mitigates the impact of TEE PKI compromise.

Forward Secrecy. When a client calls Backup, they specify a set of servers $\mathbb{S}$ and an access structure $\mathbb{A}$ over that set of servers. Notice that once enough servers in $\mathbb{S}$ go offline via the Unregister interface, it will be impossible for any sequence of Corrupt calls to let the adversary satisfy $\mathbb{A}$ (and extract the secret K via either the LeakKey interface or any sequence of recovery calls). At this point, the backup is effectively expired and completely deleted, providing forward secrecy.

Post-compromise Security. Consider a client whose backup is compromised by the adversary, meaning the adversary has corrupted $\mathsf{S} \subseteq \mathbb{S}$ such that $\mathbb{A}(\mathsf{S}) = 1$. Clearly the client can then call LeakKey to extract m. Now later, the same client initializes a new backup for a new secret m' using a different set of servers $\mathbb{S}'$ and a different access structure $\mathbb{A}'$. Assuming the adversary does not corrupt any new servers, then LeakKey will only let the adversary learn m' if $\mathbb{A}'(\mathsf{S}) = 1$. But, if a sufficient number of the server in S have gone offline, the client will eventually pick a set $\mathbb{S}'$ such that $\mathbb{S} \cap \mathbb{S}'$ is small. At the point where $\mathbb{A}'(\mathsf{S}) = 0$, the system has provided post-compromise security.

Mitigates TEE PKI failure. While $\mathcal{F}_{\mathsf{KeyBackup}}$ completely abstracts away both the TEEs and, correspondingly, the TEEs attestation PKI infrastructure, they can be seen implicitly within the ways that servers are registered with the system. If the TEE PKI infrastructure is uncompromisable, then each newly registered server *should* be honest and may only later be corrupted by a targeted attack. When the TEE PKI is compromised, we can no longer trust the attestation process, and, thus, we must assume that *all* servers that logically correspond to that TEE provider are compromised. Thus, compromising the TEE PKI infrastructure corresponds to the ideal adversary calling Corrupt for all servers (including those that are not yet online) that serve that particular type of TEE.

In the event of such a compromise, notice that these compromised offline servers do not help satisfy *any* backup access structures. The adversary must first call Register with these servers and wait for clients to make calls to Backup that use their servers. As such, this process is highly visible (i.e., every-

one will observe that many new servers are joining the system, which can possibly indicate that there is some type of catastrophic compromise) and slow, in that it requires user-driven action before the compromise is effective.

Lower order security properties. We also require security under pin reuse and no offline dictionary attacks. It is directly clear that pin reuse is not a concern, as nothing about the client's password is revealed to the adversary during recovery. Additionally, the adversary can online make online password guesses against the ideal functionality, preventing offline attacks.

Subtleties. We note that our ideal functionality doesn't explicitly specify the ways in which honest clients *should* select either their set $\mathbb{S}$ or the access structure $\mathbb{A}$. As such, its possible for to clients to choose poorly. Similarly, we don't describe a way for clients to learn or verify *which* servers are active at any given time. All of this must be handled *around* $\mathcal{F}_{\mathsf{KeyBackup}}$.

5 Construction

We provide the full construction of our password-protected secure key backup protocol $\Pi^{\mathsf{KeyBackup}}$ in Figure 3. The construction realizes the ideal functionality $\mathcal{F}^{\mathsf{KeyBackup}}$, provided in Section 4.2, in the $\mathcal{F}_{\mathsf{aoKVS}}$-hybrid algebraic group model and random oracle model. We provide the full security proof in Section 9.

Parties. Our threat model consists of a client C, a service provider SP, and a bunch of servers $\{S_i\}_i$. The adversary can potentially corrupt SP and a subset of S_i's. It can also spin up fake clients, trying to leak C's secret.

Setup Phase. During initiation, SP creates an empty blob-storage (or a map) that pairs a backup ID to the corresponding backup data (or struct). It needs to be public, that is, anyone can read this blob storage at any index. This is important, as a client, who has lost all key material, has no way to authenticate with the service provider.

When a server S_i, with trusted compute capabilities, joins, it generates a public key encryption key pair $(\mathsf{sk}_i, \mathsf{pk}_i)$ and announces the public key along with a maximum uptime t and an attestation document. Once the time is up, the server tears itself down, securely deleting the secret key. The attestation document makes sure that the public key was generated by a TEE running a predetermined binary (or VM). This binary can be published publicly for audit. We discuss the properties of our binary in Section 6.

Backup Phase. During the backup phase, the client begins with secret sharing its backup encryption key m into secret shares $\{m_1, m_2, \ldots m_n\}$ for each server $\{S_1, S_2, \ldots S_n\}$. Our construction supports any secret sharing scheme, as long as the desired access structure can be efficiently realized. The servers can be picked arbitrarily, based on users preference; if the user prefers to not keep updating the backup, it can pick servers with a larger uptime.

Then, corresponding to each server, it uniformly samples a PRF key k_i for the 2HashDH PRF, and masks the corresponding share with the PRF output on its password pwd; that is, $s_i = H_2(\mathsf{pwd}, H_1(\mathsf{pwd})^{k_i}) + m_i$. It then encrypts the PRF key k_i, along with backup ID BID and attempt limit lim, under the server S_i's announced public key to get ct_i. While BID and lim can themselves be public, including them in the ciphertext is important, to make sure they are not tampered with by SP. After repeating the process for all servers, C sends all s_i's and ct_i's to SP's blob storage, to be stored at location BID. Alternatively, a client can store it in a personal storage service, like Google Drive, that can be readily accessible in case their device is lost. This concludes the backup phase.

Recovery Phase. To recover their secret with a candidate password pwd', the client samples a random field element r and computes $\mathsf{com} = H_1(\mathsf{pwd}')^r$, and records this value on $\mathcal{F}_{\mathsf{aoKVS}}$ at BID. Then, it contacts all servers in it's original access structure, sends server S_i the backup ID BID and ciphertext ct_i from the original backup structure. The server S_i fetches (com, ℓ) from $\mathcal{F}_{\mathsf{aoKVS}}$, decrypts ct_i, checks if the BIDs (the one that is sent vs the one in ct_i) match and $\ell \leq \mathsf{lim}$, and returns back $t_i = \mathsf{com}^{k_i} = H_1(\mathsf{pwd}')^{r \cdot k_i}$. Client unblinds t_i to calculate $H_1(\mathsf{pwd}')^{k_i}$, calculates the PRF value $H_2(\mathsf{pwd}', H_1(\mathsf{pwd}')^{k_i})$ and unblinds s_i to get the secret share m_i. It reconstructs m_is to get back the secret m. In case the password is correct, i.e. $\mathsf{pwd}' = \mathsf{pwd}$, this results in the correct secret m.

Batching. Our protocol also supports coordinated batching from SP. That is, instead of clients directly contacting the servers, the clients can proxy the recovery requests through SP. This allows SP to additionally learn com^{k_i}, however, this is fine, in terms of security, due to DH-assumption. This coordination allows SP to efficiently batch requests from multiple client, and avoid client-to-server round trips from each client. Now, SP could make 1 request to each server, for say 1000 client requests, avoiding expensive round trips.

We provide the simulator and indistinguishability proof of the following theorem in Section 9:

Theorem 1. *Protocol $\Pi_{\mathsf{KeyBackup}}$ securely realizes $\mathcal{F}_{\mathsf{KeyBackup}}$ in $\mathcal{F}_{\mathsf{aoKVS}}$-hybrid, random oracle and algebraic generic model.*

5.1 Adding Social Key Recovery and Proactive Deletion

Adding our extensions into our protocol is direct. First, if we want to model social key recovery, we simply require a mechanism by which the public keys of a users' contacts can be advertised. If we aim to backup the key material for E2EE systems, this is trivial, as contacts already should be exchanging public key material (and verifying safety numbers). Thus, the protocol can remain exactly the same. Depending on the

$\Pi_{\mathsf{KeyBackup}}$

Maintenance and Setup

1. On setup, SP initiates an empty, publicly-accessible blob-store mapping backup IDs to backup structures.
2. When a server S_i joins, it samples a fresh PKE pair $(\mathsf{sk}_i, \mathsf{pk}_i)$, picks a maximum uptime $\mathtt{t}$, and broadcasts $(\mathsf{pk}_i, \mathtt{t})$.
3. S_i tears itself down, securely deleting sk_i, after $\mathtt{t}$ time has passed since provisioning.

Backup Phase

Inputs: Client C starts with a backup key m, a password pwd and an attempt limit lim

1. C generates $(\mathbb{A}, n)$-secret shares of m to get $\{m_1, m_2, \ldots m_n\}$.
2. C picks a BID such that chain with BID doesn't exist on $\mathcal{F}_{\mathsf{aoKVS}}$
3. C selects n online servers $S_1, S_2, \ldots S_n$.
4. **for** i in $[n]$:
   (a) C picks a random $k_i \leftarrow \mathbb{F}_p$ and calculate $s_i = H_2(\mathsf{pwd}, H_1(\mathsf{pwd})^{k_i}) + m_i$
   (b) C encrypts $(k_i, \mathsf{BID}, \mathsf{lim})$ under S_i's public key pk_i to get ct_i.
5. C sends $(\mathsf{BID}, s_1, s_2, \ldots, s_n, ct_1, ct_2, \ldots ct_n)$ to SP or personal storage.
6. SP stores $(s_1, s_2, \ldots, s_n, ct_1, ct_2, \ldots ct_n)$ at index BID in the blob-store (and aborts if entry at BID already exists).

Recovery Phase

Inputs: Client C starts with a password attempt pwd' and BID

1. C fetches $(s_1, s_2, \ldots, s_n, ct_1, ct_2, \ldots ct_n)$ from SP's blob-storage at index BID.
2. C samples r uniformly from $\mathbb{F}_p$, computes $\mathsf{com} = H_1(\mathsf{pwd}')^r$, and sends $(\mathsf{BID}, \mathsf{com})$ to SP
3. SP sends $(\mathsf{MakePost}, \mathsf{BID}, \mathsf{com})$ to $\mathcal{F}_{\mathsf{aoKVS}}$.
4. **for** i in $[n]$:
   (a) C sends (BID, ct_i) to S_i.
   (b) S_i sends $(\mathsf{GetPost}, \mathsf{BID})$ to $\mathcal{F}_{\mathsf{aoKVS}}$ to get back (com, ℓ)
   (c) S_i decrypts ct_i, using sk_i, into $(k_i, \mathsf{BID}', \mathsf{lim})$, and aborts if decryption fails.
   (d) S_i checks if $\ell \leq \mathsf{lim}$ and $\mathsf{BID} = \mathsf{BID}'$, and aborts otherwise
   (e) S_i sends back $t_i = \mathsf{com}^{k_i}$ to C.
   (f) C receives t_i and calculates $m_i = s_i - H_2(\mathsf{pwd}', t_i^{1/r})$.
5. C checks if the set of servers $\mathcal{R}$ that did not abort satisfies $\mathbb{A}(\mathcal{R}) = 1$, and abort otherwise.
6. C reconstructs m from received shares and aborts if reconstruction fails.

Figure 3: Password-protected Key Backup Protocol, $\Pi_{\mathsf{KeyBackup}}$.

limitation on a users' contact's devices, it may be more appropriate to remove formal guess limiting for these shares and instead enforce guess limits by (1) only allowing recovery queries to happen over NFC, so that face-to-face verification can happen, and (2) requiring the users' contact to manually approve each query through the devices user interface. Implementing these features does not require and changes to the protocol itself.

Similarly, allowing clients to proactively delete their existing keys simply requires replacing the existing CCA-secure public key encryption with a puncturable, CCA-secure public key encryption scheme [17]. Otherwise, the entire protocol remains the same.

6 Implementation

We implement a PoC of our scheme in Rust in 4400 lines of code. We use ECIES encryption scheme, from `ecies` Rust package, for CCA-secure public key encryption. This is essentially hash Elgamal, using `secp256k1` as the Elliptic Curve and `AES-256-GCM` as the symmetric cipher. We use SHA256 to realize our random oracles.

Realizing $\mathcal{F}_{\mathsf{aoKVS}}$. Again, we emphasize that the choice os implementation for $\mathcal{F}_{\mathsf{aoKVS}}$ is orthogonal to the rest of the system design. We first describe the choices we make within our implementation before discussing alternatives.

Our implementation realizes $\mathcal{F}_{\mathsf{aoKVS}}$ using Bitcoin. Specifically, during initialization, the service provider generates a new bitcoin wallet and adds some Bitcoin to it, to cover for future transaction fees. It maintains an invariant, about its wallet, that it always have one available UTXO. Whenever it needs to make a transaction with some associated data (more on the contents later), it spends the previous UTXO, to generate two outpoints: (1) an `OP_RETURN` script with the associated data, and (2) all available bitcoin (minus the fees) sent back to the wallet's address. The second outpoint becomes the new unique UTXO. Since the wallet's private key is only available with the service provider, nobody else can make these transactions.

Now, when the service provider needs to push the client query (line 3 in the recovery phase, Figure 3), it creates a transaction with associated data $(\mathsf{BID}, \mathsf{com})$, and informs all the servers about the new transaction. The servers check if the claimed transaction has indeed been made with the claimed data, using a lightweight client, and update their internal copy of the key-value store, along with a counter for number of attempts. This also means that when a new server joins, it needs to catch up from the root transaction to the current one.

The above construction requires 1 transaction per query. We can optimize using *batching*. That is, when a client makes a recovery request, the service provider stores the query in memory. When enough requests have been collected, the service provider makes a transaction with hash of all $(\mathsf{BID}_i, \mathsf{com}_i)$ tuples observed, and informs all the servers about the new transaction and data. Similar to the unbatched variant, the servers check if the hash of the transaction data matches with the claimed data, and update their internal map.

As mentioned earlier, $\mathcal{F}_{\mathsf{aoKVS}}$ could also be implemented using either traditional HSMs with hardware counter support or commodity enclaves that integrate other rollback prevention techniques. Such an instantiation would allow the system to avoid expensive interactions with a blockchain and confirmation latency. However, on the downside, this would reduce the security of the system, as now each HSM would have an individual counter, enforcing attempt limit per HSM, not backup. Depending on the access structure used by the client, this could increase the effective number of attempts. One solution to that would be to use a limit smaller than what you would use in a blockchain based system; however, even this solution is not perfect: as at the end of your effective attempt limit, you would be required to contact certain specific servers which might not be online, risking losing your secret.

Attestation Document. We configure our TEE binary to perform just three functions: (1) generate the PKE key pair, (2) on joining the pool of TEEs, fast forward to the latest transaction in our $\mathcal{F}_{\mathsf{aoKVS}}$ and verify that the transactions trace back to a hard coded root transaction; and (3) respond to recovery requests after performing necessary checks. As mentioned in Section 5, we attest to this binary and the generated public key and provide this attestation document along with the public key, publicly. Any client can audit this document against the binary hash before using the associated public key.

7 Evaluation

We deploy and benchmark our construction on server throughput (i.e. number of recovery queries supported by each server), client runtimes, and client communication sizes.

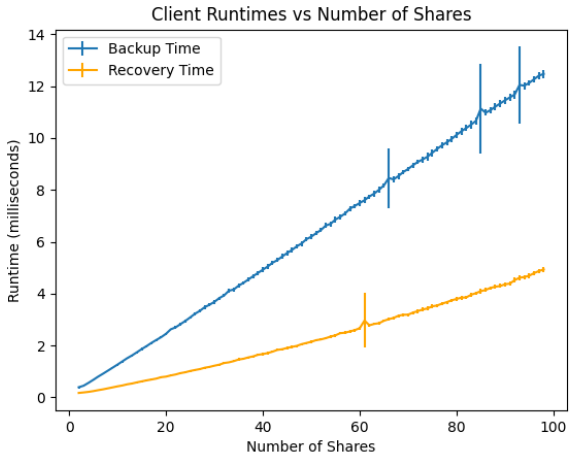
System Setup. We deploy the service provider on a Azure `D8as_v5` instance in the Central US region with 32 GiB of RAM and 8 vCPUs. In production, the service provider should be deployed on a much more powerful server, as it is a central coordination point.

We deploy three recovery servers with different confidential compute engines (i.e. TEEs), with different cloud providers:

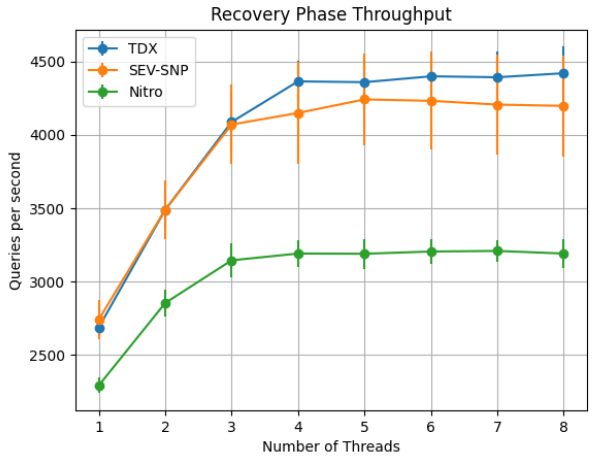
1. GCP (`c3-standard-8` instance), with Intel TDX
2. GCP (`n2d-standard-8` instance), with AMD SEV-SNP
3. AWS (`c5.2xlarge` instance), with AWS NitroTPM

All servers have 32 GiB of encrypted memory and processor with 8 vCPUs, and cost between $\$0.4 - 0.9$ per hour. We use a MacBook Air with M4 chip as the client. We deploy the append-only key-value store on Bitcoin testnet.

Client Runtimes. We benchmark our client implementation to measure the times required to (A) generate the backup,



(a) Client runtimes for backup and recovery phase across different number of servers.



(b) Throughput of different TEE types configured with different number of threads.

Figure 4: Runtime Benchmarks

	Backup Size	Recovery Query Size
$n = 3$	0.54 KB	32 bytes
$n = 100$	17.14 KB	32 bytes

Figure 5: Client side query sizes.

and (B) recovery phase (including query generation and share reconstruction). We plot runtime against the number of servers used to share the secret in Figure 4a. These numbers are averaged across 100 runs. Even with 100 servers in the access structure, our construction requires less than 13ms and 5ms for backup and reconstruction phases, respectively, on average.

Client Query Sizes. We summarize client's backup and recovery query sizes in Figure 5. For a single backup to n servers, the backup contains n ciphertexts and n field elements (for shamir secret shares masked with PRF output). Each scalar takes 32 bytes (256 bits) and each ciphertext, in our implementation, requires 137 bytes. The entire backup struct, for $n = 100$, along with all metadata, requires 17551 bytes ($\approx$ 17.14 KB). For $n = 3$, the backup size is 559 bytes. Roughly, for n servers, the backup size is $33 + 175n$ bytes.

For recovery phase, the client only needs to send a single group element to the service provider, consisting of 32 bytes.

Throughput. To benchmark the throughput of the servers during recovery phase, consider the following experimental setup: the service provider hosts 1000 backups for 1000 clients. Each backup is configured to require responses from all three servers. Then, during recovery phase, the service provider collects recovery queries from all clients. Once all requests are collected, the service provider makes a Bitcoin transaction. Once the transaction propagates, the service provider contacts all of the servers to get back their recovery responses. For all TEE types (i.e. TDX, SEV-SNP and NitroTPM), we measure the number of queries supported by the server per second, in this setup. We run this experiment, 100 times, against the TEE server configured with different number of threads and present the averaged numbers in Figure 4b. Our results do not include the time required to confirm the transaction.

Intel TDX and AMD SEV-SNP provide comparable performance, peaking at 4420 and 4242 queries per second, respectively, while AWS NitroTPM has a slightly lower peak throughput of 3209 queries per second.

8 Related Works

Password-Protected Secret Sharing Our work is deeply inspired by the works on *Password Protected Secret Sharing*, first introduced by Bagherzandi et al [4]. There has been a significant line of work on improving the efficiency [1, 7, 19, 21] and providing formal security analysis [20]. Our work builds on the ideas in these works by confronting the real-world challenges that come with deploying these systems and providing security properties like forward secrecy, post-compromise security, and robustness against PKI compromise.

Rollback Prevention in Enclaves. Nimble [3] and ROTE [29] both consider augmenting TEEs to detect rollback attacks. Nimble achieves this by building a ledger, while ROTE uses a novel distributed algorithm to achieve the same. These works, however, do not consider how to maintain security when the trusted hardware itself can be compromised.

Kaptchuk et al [24] formalize a way to augment a stateless trusted execution environment with a ledger, to allow the hardware with limited secure memory to maintain arbitrary

states and prevent replay/rollback attacks. They also consider limiting password attempts using this system.

Secure Key Recovery using Trusted Hardware. There has also been work on using trusted execution environments to implement secret key recovery protocols [2], with most popular implementations of such being by WhatsApp [40] and Apple [15, 16]. These works, however, require HSMs to work and to the best of our knowledge, assume un-breakability of a single HSM (or respective trusted hardware).

To overcome this issue, SafetyPin [12] explored the idea of splitting the secret among low-cost HSMs while limiting password guesses counters. At its core, SafetyPin clients hash their password (and some public parameters) to deterministically select a subset of servers and encrypt secret shares of their message to these servers. This results in a non-interactive backup procedure similar to ours, but does not make use of the OPRF template that has become standard. Additionally, SafetyPin explores using puncturable encryption to get forward secrecy in a very similar manner to our own work. Finally, they build a distributed log to limit password attempts. While the ideas presented in SafetyPin are elegant, their construction suffers from two deployability challenges. First, they leverage random auditing to ensure the password log is being constructed honestly, which is risky when the auditors could be cloud-deployed TEEs, as we consider in our work. Second, their construction provides no post-compromise security, forward secrecy requires clients to actively puncture backups, and ensuring password-reuse is safe requires *active* checking on the part of clients.

SVR3 [11] is a more recent work which was partially deployed by Signal. Similar to our work, they also make use of OPRF-based PPSS protocol with counters to implement attempt limits. Instead of public ledgers, they make use of a modified version of Raft to implement counters. A key limitation of SVR3 is that it is not designed with forward secrecy or post compromise security in mind. And does not protect against TEE PKI Compromise.After publication, the authors of SVR3 added a note about key rotation, in response to identified issues. It is unclear how it would be implemented at scale, specifically because SVR3 requires interaction for backups.

Our construction builds on some of the best ideas developed in SafetyPin and SVR3, while offering improved deployability and security. Specifically, our non-interactive backup scheme is similar, in spirit, to the one considered in SafetyPin, but uses the OPRF approach considered in SVR3 (among others). Similarly, SafetyPin leverages the idea that each HSM should use different key material, but does not consider using commodity cloud-based TEEs, which SVR3 does. Neither work first-order considers key rotation.

We note that neither SafetyPin nor SVR3 provide a proofs of security or formal definitions. Davies et al. [40] provide a security proof for WhatsApp's backup system. For a detailed survey of recovery systems, we refer the reader to [6].

References

[1] Michel Abdalla, Mario Cornejo, Anca Nitulescu, and David Pointcheval. Robust password-protected secret sharing. Cryptology ePrint Archive, Paper 2016/123, 2016. URL: https://eprint.iacr.org/2016/123, doi:10.1007/978-3-319-45741-34.

[2] David Adei, Chris Orsini, Alessandra Scafuro, and Tanner Verber. How to recover a cryptographic secret from the cloud. In Chun-Ying Huang, Jyh-Cheng Chen, Shiuh-Pyng Shieh, David Lie, and Véronique Cortier, editors, *ACM CCS 2025*, pages 1814–1828. ACM Press, October 2025. doi:10.1145/3719027.3765127.

[3] Sebastian Angel, Aditya Basu, Weidong Cui, Trent Jaeger, Stella Lau, Srinath Setty, and Sudheesh Singanamalla. Nimble: Rollback protection for confidential cloud services (extended version). Cryptology ePrint Archive, Paper 2023/761, 2023. URL: https://eprint.iacr.org/2023/761.

[4] Ali Bagherzandi, Stanislaw Jarecki, Nitesh Saxena, and Yanbin Lu. Password-protected secret sharing. In *ACM CCS*, 2011.

[5] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from Bitcoin. In *2014 IEEE Symposium on Security and Privacy*, pages 459–474. IEEE Computer Society Press, May 2014. doi:10.1109/SP.2014.36.

[6] Jenny Blessing, Daniel Hugenroth, Ross Anderson, and Alastair Beresford. Sok: Web authentication and recovery in the age of end-to-end encryption. *Proceedings on Privacy Enhancing Technologies*, 2025:560–589, 07 2025. doi:10.56553/popets-2025-0113.

[7] Jan Camenisch, Anja Lehmann, Anna Lysyanskaya, and Gregory Neven. Memento: How to reconstruct your secrets from a single password in a hostile environment. Cryptology ePrint Archive, Paper 2014/429, 2014. URL: https://eprint.iacr.org/2014/429.

[8] Melissa Chase, Apoorvaa Deshpande, Esha Ghosh, and Harjasleen Malvai. SEEMless: Secure end-to-end encrypted messaging with less trust. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 1639–1656. ACM Press, November 2019. doi:10.1145/3319535.3363202.

[9] Panagiotis Chatzigiannis, Konstantinos Chalkias, Aniket Kate, Easwar Vivek Mangipudi, Mohsen Minaei, and Mainack Mondal. SoK: Web3 recovery mechanisms. Cryptology ePrint Archive, Report 2023/1575, 2023. URL: https://eprint.iacr.org/2023/1575.

[10] Jalen Chuang, Alex Seto, Nicolas Berrios, Stephan van Schaik, Christina Garman, and Daniel Genkin. Tee.fail: Breaking trusted execution environments via ddr5 memory bus interposition. In *47th IEEE Symposium on Security and Privacy (IEEE S&P '26)*. IEEE Computer Society, 2026. URL: https://tee.fail.

[11] Graeme Connell, Vivian Fang, Rolfe Schmidt, Emma Dauterman, and Raluca Ada Popa. Secret key recovery in a Global-Scale End-to-End encryption system. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 703–719, Santa Clara, CA, July 2024. USENIX Association. URL: https://www.usenix.org/conference/osdi24/presentation/connell.

[12] Emma Dauterman, Henry Corrigan-Gibbs, and David Mazières. Safetypin: encrypted backups with human-memorable secrets. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation*, OSDI'20, USA, 2020. USENIX Association.

[13] Yevgeniy Dodis and Daniel Jost. Compact key storage in the standard model. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024, Part I*, volume 15364 of *LNCS*, pages 444–475. Springer, Cham, December 2024. doi:10.1007/978-3-031-78011-0_15.

[14] Yevgeniy Dodis, Daniel Jost, and Antonio Marcedone. Compact key storage - A modern approach to key backup and delegation. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part II*, volume 14921 of *LNCS*, pages 75–109. Springer, Cham, August 2024. doi:10.1007/978-3-031-68379-4_3.

[15] Matthew Green. Is Apple's Cloud Key Vault a crypto backdoor? Available at https://blog.cryptographyengineering.com/2016/08/13/is-apples-cloud-key-vault-crypto/, 2016.

[16] Matthew Green. Why encrypted backup is so important. Available here, 2022.

[17] Matthew D. Green and Ian Miers. Forward secure asynchronous messaging from puncturable encryption. In *2015 IEEE Symposium on Security and Privacy*, pages 305–320. IEEE Computer Society Press, May 2015. doi:10.1109/SP.2015.26.

[18] Hossein Hafezi, Alireza Shirzad, Benedikt Bünz, and Joseph Bonneau. IronDict: Transparent dictionaries from polynomial commitments. Cryptology ePrint Archive, Report 2025/1580, 2025. URL: https://eprint.iacr.org/2025/1580.

[19] Stanislaw Jarecki, Aggelos Kiayias, and Hugo Krawczyk. Round-optimal password-protected secret sharing and T-PAKE in the password-only model. In Palash Sarkar and Tetsu Iwata, editors, *ASIACRYPT 2014, Part II*, volume 8874 of *LNCS*, pages 233–253. Springer, Berlin, Heidelberg, December 2014. doi:10.1007/978-3-662-45608-8_13.

[20] Stanislaw Jarecki, Aggelos Kiayias, Hugo Krawczyk, and Jiayu Xu. Highly-efficient and composable password-protected secret sharing (or: How to protect your bitcoin wallet online). In *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 276–291, 2016. doi:10.1109/EuroSP.2016.30.

[21] Stanisław Jarecki, Aggelos Kiayias, Hugo Krawczyk, and Jiayu Xu. Toppss: cost-minimal password-protected secret sharing based on threshold oprf. In *International Conference on Applied Cryptography and Network Security*, pages 39–58. Springer, 2017.

[22] Stanislaw Jarecki, Hugo Krawczyk, and Jiayu Xu. OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 456–486. Springer, Cham, April / May 2018. doi:10.1007/978-3-319-78372-7_15.

[23] Gabriel Kaptchuk, Matthew Green, and Ian Miers. Giving state to the stateless: Augmenting trustworthy computation with ledgers. In *NDSS 2019*. The Internet Society, February 2019. doi:10.14722/ndss.2019.23060.

[24] Gabriel Kaptchuk, Ian Miers, and Matthew Green. Giving state to the stateless: Augmenting trustworthy computation with ledgers. In *NDSS*, 2019.

[25] Aniket Kate, Pratyay Mukherjee, Hamza Saleem, Pratik Sarkar, and Bhaskar Roberts. ANARKey: A new approach to (socially) recover keys. Cryptology ePrint Archive, Report 2025/551, 2025. URL: https://eprint.iacr.org/2025/551.

[26] Julia Len, Melissa Chase, Esha Ghosh, Kim Laine, and Radames Cruz Moreno. OPTIKS: An optimized key transparency system. In Davide Balzarotti and Wenyuan Xu, editors, *USENIX Security 2024*. USENIX Association, August 2024. URL: https://www.usenix.org/conference/usenixsecurity24/presentation/len.

[27] Harjasleen Malvai, Lefteris Kokoris-Kogias, Alberto Sonnino, Esha Ghosh, Ercan Oztürk, Kevin Lewi, and Sean F. Lawlor. Parakeet: Practical key transparency for end-to-end encrypted messaging. In *NDSS 2023*. The Internet Society, February 2023.

[28] Easwar Vivek Mangipudi, Panagiotis Chatzigiannis, Konstantinos Chalkias, Aniket Kate, Mohsen Minaei, and Mainack Mondal. Web3 recovery mechanisms and user preferences. Cryptology ePrint Archive, Report 2025/1687, 2025. URL: https://eprint.iacr.org/2025/1687.

[29] Sinisa Matetic, Mansoor Ahmed, Kari Kostiainen, Aritra Dhar, David Sommer, Arthur Gervais, Ari Juels, and Srdjan Capkun. ROTE: Rollback protection for trusted execution. In *26th USENIX Security Symposium (USENIX Security 17)*, pages 1289–1306, Vancouver, BC, August 2017. USENIX Association. URL: https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/matetic.

[30] Marcela S. Melara, Aaron Blankstein, Joseph Bonneau, Edward W. Felten, and Michael J. Freedman. CONIKS: Bringing key transparency to end users. In Jaeyeon Jung and Thorsten Holz, editors, *USENIX Security 2015*, pages 383–398. USENIX Association, August 2015. URL: https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/melara.

[31] Ian Miers, Christina Garman, Matthew Green, and Aviel D. Rubin. Zerocoin: Anonymous distributed E-cash from Bitcoin. In *2013 IEEE Symposium on Security and Privacy*, pages 397–411. IEEE Computer Society Press, May 2013. doi:10.1109/SP.2013.34.

[32] Shailesh Mishra, Simone Colombo, Pasindu Tennage, Martin Burkhart, and Bryan Ford. Towards the ideals of self-recovery and metadata privacy in social vault recovery, 2025. URL: https://arxiv.org/abs/2507.19484, arXiv:2507.19484.

[33] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. Available at https://bitcoin.org/bitcoin.pdf, 2008.

[34] Alex Seto, Oytun Kuday Duran, Samy Amer, Jalen Chuang, Stephan van Schaik, Daniel Genkin, and Christina Garman. Wiretap: Breaking server sgx via dram bus interposition. In *2025 SIGSAC Conference on Computer and Communications Security (CCS '25)*. Association for Computing Machinery, 2025. URL: https://wiretap.fail.

[35] Adi Shamir. How to share a secret. *Communications of the Association for Computing Machinery*, 22(11):612–613, November 1979. doi:10.1145/359168.359176.

[36] Jo Van Bulck, Marina Minkin, Ofir Weisse, Daniel Genkin, Baris Kasikci, Frank Piessens, Mark Silberstein, Thomas F. Wenisch, Yuval Yarom, and Raoul Strackx.

Foreshadow: Extracting the keys to the Intel SGX kingdom with transient out-of-order execution. In *Proceedings of the 27th USENIX Security Symposium*. USENIX Association, August 2018.

[37] Jo Van Bulck, Daniel Moghimi, Michael Schwarz, Moritz Lipp, Marina Minkin, Daniel Genkin, Yarom Yuval, Berk Sunar, Daniel Gruss, and Frank Piessens. LVI: Hijacking Transient Execution through Microarchitectural Load Value Injection. In *41th IEEE Symposium on Security and Privacy (S&P'20)*, 2020.

[38] Stephan van Schaik, Andrew Kwong, Daniel Genkin, and Yuval Yarom. SGAxe: How SGX fails in practice. https://sgaxeattack.com/, 2020.

[39] Stephan van Schaik, Alex Seto, Thomas Yurek, Adam Batori, Bader AlBassam, Christina Garman, Daniel Genkin, Andrew Miller, Eyal Ronen, and Yuval Yarom. SoK: SGX.Fail: How stuff get eXposed. 2022.

[40] WhatsApp. Security of end-to-end encrypted backups. https://www.whatsapp.com/security/WhatsApp_Security_Encrypted_Backups_Whitepaper.pdf, 2021. Whitepaper.

9 Security Proof

Theorem 1. *Protocol $\Pi_{\mathsf{KeyBackup}}$ securely realizes $\mathcal{F}_{\mathsf{KeyBackup}}$ in $\mathcal{F}_{\mathsf{aoKVS}}$-hybrid, random oracle and algebraic generic model.*

Proof. To prove the security of the protocol, we build a simulator Sim, such that, for all real world adversaries $\mathcal{A}$, the view of $\mathcal{A}$ in real world is computationally indistinguishable with the simulated view generated by Sim.

We split any execution in ideal or real world into three disjoint steps: an honest client makes a backup, an honest client makes a recovery attempt, and the adversary/simulator corrupts a server that was previously honest. After each of these interactions, $\mathcal{A}$ can launch an attack phase where it can perform some local computations, provide wrong outputs to recovery attempts, or pose as a malicious client to make some dummy backups or recovery attempts. For our protocol to be secure, we need to prove that for any execution of these steps, the joint distribution of the output of honest parties and the view of adversary is computationally indistinguishable between real and ideal worlds.

To this end, we explain our simulation strategy in all three cases. We provide the full simulator in Figure 6.

Step Type 1: An Honest Client makes a Backup. In the real world, $\mathcal{A}$ receives (through the blob-store), the post which consists of $(s_1, s_2, \ldots, s_n, ct_1, ct_2, \ldots ct_n)$, where $s_i = H_2(\mathsf{pwd}, H_1(\mathsf{pwd})^{k_i}) + m_i$ generated by an honest client. In the ideal world, $\mathcal{A}$ instead receives this simulated message $(s'_1, s'_2, \ldots, s'_n, ct'_1, ct'_2, \ldots ct'_n)$ generated by Sim. In case the backup is already compromised (i.e. $\mathbb{A}(\mathsf{Corr} \cap \mathbb{S}) = 1$), the distributions of these backups are identical, as the process to generate these messages is identical, hence, computationally indistinguishable. When the backup is not compromised, (i.e. $\mathbb{A}(\mathsf{Corr} \cap \mathbb{S}) = 0$), all s'_i are uniformly sampled from $\mathbb{G}$; rest of the message still follows the same process. As there is no output in this step for the honest client, it is sufficient to prove indistinguishability between these messages in real and ideal world, when $\mathcal{A}$ corrupts below the threshold. We prove this based on the security properties of public key encryption, DH assumption, and $(\mathbb{A}, n)$-secret sharing scheme.

Lemma 1. *For an uncompromised backup (i.e. $\mathbb{A}(\mathsf{Corr} \cap \mathbb{S}) = 0$), the distribution of the ledger post $(s_1, s_2, \ldots s_n, ct_1, ct_2, \ldots ct_n)$ is indistinguishable between ideal and real worlds.*

Proof. We use hybrid argument to prove this. Without loss of generality, assume the last t of n servers are compromised; that is, for $i \in [n-t], S_i \notin \mathsf{Corr}$ and for $i \in [n-t+1, n], S_i \in \mathsf{Corr}$, while still maintaining that $\mathbb{A}(\mathsf{Corr} \cap \mathbb{S}) = 0$. As a result, the adversary knows the decryption keys for ciphertexts $ct_{n-t+1}, ct_{n-t+2}, \ldots ct_n$. Consider $n-t+2$ hybrid executions: $\mathcal{H}_0, \mathcal{H}_1, \ldots \mathcal{H}_{n-t+1}$. $\mathcal{H}_0$ is the real world. For $i \in [n-t]$, in $\mathcal{H}_i$, $\mathcal{S}$ samples first i s_js uniformly. $\mathcal{H}_{n-t+1}$ is identical to ideal world, i.e., all s_i's are uniformly sampled.

We now prove pairwise indistinguishability of hybrids:

1. $\underline{\mathcal{H}_{i-1} \text{ and } \mathcal{H}_i \text{ for } i \in [n-t]}$: These two hybrids only differ in s_i; it is equal to $H_2(\mathsf{pwd}, H_1(\mathsf{pwd})^{k_i}) + m_i$ in $\mathcal{H}_{i-1}$, and uniformly sampled in $\mathcal{H}_i$. We introduce a new hybrid $\mathcal{H}'_{i-1}$ between $\mathcal{H}_{i-1}$ and $\mathcal{H}_i$. In $\mathcal{H}'_{i-1}$ we uniformly sample k_i twice: Specifically we compute s_i, ct_i as follows:

$$s_i = H_2(\mathsf{pwd}, H_1(\mathsf{pwd})^{k_i}) + m_i$$

$$ct_i = \mathsf{Encrypt}_{\mathsf{pk}}(k'_i, \mathsf{BID}, \mathsf{lim})$$

for $k_i, k'_i \leftarrow\$ \mathbb{F}_p$. For any recovery requests with com for this backup, the simulator responds with com^{k_i}.

$\underline{\mathcal{H}_{i-1} \text{ and } \mathcal{H}'_{i-1}}$. Let's assume for contradiction that the adversary $\mathcal{A}$ can distinguish between $\mathcal{H}_{i-1}$ and $\mathcal{H}'_{i-1}$ for some client input $(\mathsf{BID}, \mathsf{pwd}, m, \mathsf{lim})$. Using $\mathcal{A}$, we can build a distinguisher $\mathcal{D}_i^{\mathsf{CCA}}$ that can break the CCA-security of PKE.

Distinguisher $\mathcal{D}_i^{\mathsf{CCA}}$

(a) Receive pk from the challenger and announce pk as the public key of S_i.

(b) For other $n-1$ servers S_j, generate keypairs $(\mathsf{pk}_j, \mathsf{sk}_j)$ and announce pk_j.

(c) Randomly samples two messages M_0, M_1 from $\mathbb{F}_p$ and sends $(M_0, \mathsf{BID}, \mathsf{lim})$ and

$\mathsf{Sim}_{\mathsf{KeyBackup}}$

Backup Phase

On receiving $(\mathsf{Backup}, \mathsf{BID}, \mathsf{CBID}, \mathbb{S}, \mathbb{A}, \mathsf{lim})$ from $\mathcal{F}_{\mathsf{KeyBackup}}$: ▷ Backup initiated by honest client

1. Set $n \leftarrow |\mathbb{S}|$
2. If $\mathbb{A}(\mathsf{Corr} \cap \mathbb{S}) = 1$
   (a) Send $(\mathsf{LeakKey}, \mathsf{BID}, \mathsf{CBID})$ to $\mathcal{F}_{\mathsf{KeyBackup}}$ to get back (m, pwd)
   (b) Secret share m using $(\mathbb{A}, n)$-scheme to get $\{m_1, m_2, \ldots m_n\}$.
3. **for** i in $[n]$:
   (a) Pick a random $K_i \leftarrow \mathbb{F}_p$
   (b) If $\mathbb{A}(\mathsf{Corr} \cap \mathbb{S}) = 1$, set $s_i = H_2(\mathsf{pwd}, H(\mathsf{pwd})^{K_i}) + m_i$, otherwise sample $s_i \leftarrow \mathbb{G}$
   (c) Encrypt $(K_i, \mathsf{BID}, \mathsf{lim})$ under S_i's public key to get ct_i.
4. Publish $(s_1, s_2, \ldots, s_n, ct_1, ct_2, \ldots ct_n)$ to the public blob storage at BID.

Recovery Phase

On receiving $(\mathsf{RecoveryStart}, \mathsf{BID}, \mathsf{CBID}, \mathsf{attempt})$ from $\mathcal{F}_{\mathsf{KeyBackup}}$: ▷ Recovery initiated by honest client

1. Pick random $\mathsf{com} \leftarrow \mathbb{G}$ and post it on $\mathcal{F}_{\mathsf{aoKVS}}$ at BID
2. **for** i in $[|\mathsf{Corr} \cap \mathbb{S}|]$:
   (a) Let S_i be the i-th server in set $\mathsf{Corr} \cap \mathbb{S}$
   (b) Send (BID, ct_i), authenticated to S_i, to $\mathcal{A}$, to get back $t_i \in \mathbb{G}$
   (c) if $t_i = \mathsf{com}^{K_i}$, send $(\mathsf{AllowRecovery}, S_i, \mathsf{BID}, \mathsf{CBID}, \mathsf{attempt})$ to $\mathcal{F}_{\mathsf{KeyBackup}}$.
3. Send $(\mathsf{FinalizeRecovery}, \mathsf{BID}, \mathsf{CBID}, \mathsf{attempt})$ to $\mathcal{F}_{\mathsf{KeyBackup}}$

Server Management

Subroutine $\mathsf{Reprogram}(\mathsf{BID}, m, \mathsf{pwd})$

1. Collect the set T of servers S_i, for which H_2 has been queried at $(\mathsf{pwd}, H(\mathsf{pwd})^{K_i})$.
2. $\mathbb{A}(T) = 1$, fail simulation.
3. For all $i \in T$, set $m_i = s_i - H_2(\mathsf{pwd}, H(\mathsf{pwd})^{K_i})$
4. Sample set $\{m_i\}_{i \notin T}$ from the marginal distribution of sharing of m with fixed $\{m_i\}_{i \in T}$
5. For all $i \notin T$, program $H_2(\mathsf{pwd}, H(\mathsf{pwd})^{K_i}) = s_i - m_i$.

On receiving $(\mathsf{Corrupt}, S)$ from $\mathcal{A}$:

1. Iterate over all $(\mathsf{BID}, \mathsf{CBID}, \mathbb{S}, \mathbb{A})$ observed via a Backup call till now. If $\mathbb{A}(\mathbb{S} \cap \mathsf{Corr}) = 0$ and $\mathbb{A}(\mathbb{S} \cap (\mathsf{Corr} \cup \{S\})) = 1$:
   (a) Send $(\mathsf{LeakKey}, \mathsf{BID}, \mathsf{CBID})$ to $\mathcal{F}_{\mathsf{KeyBackup}}$ to get back (m, pwd)
   (b) Invoke subroutine $\mathsf{Reprogram}(\mathsf{BID}, \mathsf{m}, \mathsf{pwd})$
2. Send S's PKE secret key to $\mathcal{A}$
3. Add S to Corr and send $(\mathsf{Corrupt}, S)$ to $\mathcal{F}_{\mathsf{KeyBackup}}$

Figure 6: Simulator for Password-protected Key Backup Protocol, $\mathsf{Sim}_{\mathsf{KeyBackup}}$. Continued in Figure 7

$\mathsf{Sim}_{\mathsf{KeyBackup}}$ (Continued)

Corrupt Client

On receiving $(\mathsf{MakePost}, \mathsf{BID}, \mathsf{com})$ from $\mathcal{F}_{\mathsf{aoKVS}}$:

1. If BID was observed through one of Backup calls: ▷ Adversary trying to use up an attempt of an honest backup
   (a) Find pwd, g, r (observed through AGM and RO H) such that $g = H(\mathsf{pwd})$ and $\mathsf{com} = g^r$
   (b) If such a tuple is found:
      i. Send $(\mathsf{InitRecovery}, \mathsf{BID}, \mathsf{CBID}, \mathsf{pwd})$ to $\mathcal{F}_{\mathsf{KeyBackup}}$
      ii. For $S_i \in \mathsf{Corr} \cap \mathbb{S}$, send $(\mathsf{AllowRecovery}, S_i, \mathsf{BID}, \mathsf{CBID}, \mathsf{attempt})$ to $\mathcal{F}_{\mathsf{KeyBackup}}$
      iii. Send $(\mathsf{FinalizeRecovery}, \mathsf{BID}, \mathsf{CBID}, \mathsf{attempt})$ to get back $(\mathsf{Success}, m)$ or $(\mathsf{Failure}, 0^\lambda)$.
      iv. In case of success, invoke subroutine $\mathsf{Reprogram}(\mathsf{BID}, \mathsf{m}, \mathsf{pwd})$

On receiving (BID, ct_i) authenticated to honest S_i from $\mathcal{A}$:

1. Send $(\mathsf{GetPost}, \mathsf{BID})$ to $\mathcal{F}_{\mathsf{aoKVS}}$ to get back (com, ℓ)
2. Decrypt ct_i into $(K_i, \mathsf{BID}', \mathsf{lim})$, and abort if decryption fails.
3. Check if ℓ is smaller than lim and $\mathsf{BID} = \mathsf{BID}'$, and abort otherwise.
4. Send back $t_i = \mathsf{com}^{K_i}$ to $\mathcal{A}$.

Figure 7: Simulator for Password-protected Key Backup Protocol, $\mathsf{Sim}_{\mathsf{KeyBackup}}$, continued.

$(M_1, \mathsf{BID}, \mathsf{lim})$ to challenger to get back ct.

(d) For all $j \in [n] - \{i\}$, uniformly samples k_j. Sets $k_i = M_0$

(e) $\{m_1, m_2, \ldots m_n\} \leftarrow\$ \mathsf{Share}(m)$

(f) For $j \in [i-1]$, sample s_j uniformly.

(g) For $j \in [i, n]$, set $s_j = H_2(\mathsf{pwd}, H_1(\mathsf{pwd})^{k_j}) + m_i$

(h) For $j \in [n] - \{i\}$, set $ct_j \leftarrow \mathsf{Encrypt}_{\mathsf{pk}_j}(k_j, \mathsf{BID}, \mathsf{lim})$. Set $ct_i = ct$

(i) Create a backup $(s_1, s_2, \ldots, s_n, ct_1, ct_2, \ldots ct_n)$

(j) When $\mathcal{A}$ makes a recovery request with S_i for (BID', ct_i) with com:
   i. if $\mathsf{BID} = \mathsf{BID}'$, respond with $t_i = \mathsf{com}^{k_i}$
   ii. if $\mathsf{BID} \neq \mathsf{BID}'$, use decryption oracle to decrypt required ct_i to get M, return com^M. Send $\perp$ if oracle sends $\perp$.

(k) When $\mathcal{A}$ outputs b, output b

In case challenger picks $(M_0, \mathsf{BID}, \mathsf{lim})$, the post follows distribution from $\mathcal{H}_{i-1}$. If challenger picks $M_1, \mathsf{BID}, \mathsf{lim}$, the post follows distribution from $\mathcal{H}'_{i-1}$. Therefore the advantage $\mathcal{D}_i$ has in the CCA-PKE game is equal to the advantage $\mathcal{A}$ has in distinguishing between $\mathcal{H}_{i-1}$ and $\mathcal{H}'_{i-1}$. Therefore $\mathcal{A}$ cannot distinguish between the hybrids.

$\mathcal{H}'_{i-1}$ and $\mathcal{H}_i$. To prove indistinguishability between $\mathcal{H}'_{i-1}$ and $\mathcal{H}_i$, we use the fact that $F(k,x) = H_1(x)^k$ is a PRF [20]. Therefore, we show that the existence of a adversary $\mathcal{A}$ that distinguishes between $\mathcal{H}'_{i-1}$ and $\mathcal{H}_i$ for some client input $(\mathsf{BID}, \mathsf{pwd}, m, \mathsf{lim})$ with non-negligible advantage implies the existence of a distinguisher $\mathcal{D}_i^{\mathsf{PRF}}$ that can win the PRF game (i.e. can distinguish between a random function or PRF given as oracle) for the PRF $F(k,x) = H(x)^k$.

Distinguisher $\mathcal{D}_i^{\mathsf{PRF}}$

(a) Sends pwd to the challenger to get back r.

(b) For all $j \in [n]$, uniformly samples k_j.

(c) $\{m_1, m_2, \ldots m_n\} \leftarrow\$ \mathsf{Share}(m)$

(d) For $j \in [i-1]$, sample s_j uniformly.

(e) Sets $s_i = H_2(\mathsf{pwd}, r) + m_i$.

(f) For $j \in [i+1, n]$, set $s_j = H_2(\mathsf{pwd}, H_1(\mathsf{pwd})^{k_j}) + m_j$.

(g) For $j \in [n]$, sets $ct_j \leftarrow \mathsf{Encrypt}_{\mathsf{pk}_j}(k_j, \mathsf{BID}, \mathsf{lim})$

(h) Create a backup $(s_1, s_2, \ldots, s_n, ct_1, ct_2, \ldots ct_n)$

(i) When $\mathcal{A}$ makes a recovery request with S_i for BID with com:
   i. Find pwd', g, a (observed through AGM and RO H_1) such that $g = H(\mathsf{pwd}')$ and $\mathsf{com} = g^a$
   ii. Send pwd' to challenger to get back p

iii. Send back p^a to $\mathcal{A}$

(j) When $\mathcal{A}$ outputs b, output b

$\mathcal{H}'_{i-1}$ and $\mathcal{H}_i$ only differ in s_i. In case the challenger decides to use the PRF, $s_i = H_2(\mathsf{pwd}, H_1(\mathsf{pwd})^{k'_i}) + m_i$, for randomly sampled k'_i and hence the post would follow distribution of $\mathcal{H}'_{i-1}$. In case challenger uses random function, r is uniform, and so is s_i. Therefore, the post follows distribution of $\mathcal{H}_i$. Therefore the advantage $\mathcal{D}_i$ has in the PRF game is equal to the advantage $\mathcal{D}'_i$ has in distinguishing between $\mathcal{H}'_{i-1}$ and $\mathcal{H}_i$. Therefore a distringuisher $\mathcal{D}_i$ cannot exist.

2. $\mathcal{H}_{n-t}$ and $\mathcal{H}_{n-t+1}$: These two hybrids differ in last t s_i's; additionally, in this case, the adversary additionally knows the decryption keys for ct_i. We show $\mathcal{H}_{n-t}$ and $\mathcal{H}_{n-t+1}$ are computationally indistinguishable by building a distinguisher $\mathcal{D}^{\mathsf{Share}}$ which breaks the uniformity property of $(\mathbb{A}, n)$ sharing scheme, using $\mathcal{A}$ that can distinguish between $\mathcal{H}_{n-t}$ and $\mathcal{H}_{n-t+1}$ for some client input $(\mathsf{BID}, \mathsf{pwd}, m, \mathsf{lim})$.

Distinguisher $\mathcal{D}^{\mathsf{Share}}$

(a) Send m to challenger to get back t $(\mathbb{A}, n)$-secret shares $\{m_{n-t+1}, m_{n-t+2}, \ldots m_n\}$ from the challenger.

(b) For all $i \in [n]$, uniformly samples k_i.

(c) For $i \in [n-t]$, samples s_i uniformly from the domain of i-th share.

(d) For $i \in [n-t+1, n]$, sets $s_i = H_2(\mathsf{pwd}, H_1(\mathsf{pwd})^{k_i}) + m_i$

(e) For $i \in [n]$, sets $ct_i \leftarrow \mathsf{Encrypt}_{\mathsf{pk}_i}(k_i, \mathsf{BID}, \mathsf{lim})$

(f) Create a backup $(s_1, s_2, \ldots, s_n, ct_1, ct_2, \ldots ct_n)$

(g) Respond to $\mathcal{A}$ queries as usual.

(h) When $\mathcal{A}$ outputs b, output b

When the challenger samples $\{m_{n-t+1}, m_{n-t+2}, \ldots m_n\}$ uniformly, the distribution of post that $\mathcal{D}^{\mathsf{Share}}$ makes is identical to $\mathcal{H}_{n-t+1}$, as we mask $H_2(\mathsf{pwd}, H_1(\mathsf{pwd})^{k_i})$ with uniform values, resulting in all s_i's being uniform. When challenger provides real shares, the distribution of post $\mathcal{D}^{\mathsf{Share}}$ makes is identical to $\mathcal{H}_{n-t}$. Therefore, the probability of $\mathcal{D}^{\mathsf{Share}}$ winning is exactly equal to the probability of $\mathcal{A}$ distinguishing $\mathcal{H}_{n-t}$ and $\mathcal{H}_{n-t+1}$. Therefore $\mathcal{A}$ cannot distinguish between the hybrids.

□

Step Type 2: An Honest Client makes a Recovery. Unlike case 1, interaction of this type is not the first message about the backup in question; any number or type of interactions can happen before this interaction, which are dependent on this backup. Let $\mathcal{M}^{\mathsf{Real}}$ and $\mathcal{M}^{\mathsf{Ideal}}$ be the set of messages received by $\mathcal{A}$ and honest outputs, in real and ideal world resp., due to these prior interactions, relevant to the backup in question. Due to our inductive argument, $\mathcal{M}^{\mathsf{Real}}$ is computationally indistinguishable from $\mathcal{M}^{\mathsf{Ideal}}$. In the current interaction, the adversary learns com, which is randomly sampled group element g in the ideal world and $H(\mathsf{pwd})^r$ in the real world. We show that joint view generated by $\mathcal{M}^{\mathsf{Real}}, H(\mathsf{pwd})^r$ is computationally indistinguishable from $\mathcal{M}^{\mathsf{Ideal}}, g$.

We use hybrid argument with three hybrids: $\mathcal{H}_0$, $\mathcal{H}_1$ and $\mathcal{H}_2$. $\mathcal{H}_0$ is the joint distribution in real world, $\mathcal{H}_2$ is the joint distribution in the ideal world, and $\mathcal{H}_1$ is the joint distribution $\mathcal{M}^{\mathsf{Real}}, g$. Indistinguishability is trivial:

1. $\mathcal{H}_0$ *and* $\mathcal{H}_1$: As $H_1(\mathsf{pwd})$ is a generator of $\mathbb{G}$, $H_1(\mathsf{pwd})^r$ is uniform in $\mathbb{G}$. Therefore, $\mathcal{H}_0$ and $\mathcal{H}_1$ are identical.

2. $\mathcal{H}_1$ *and* $\mathcal{H}_2$: This directly follows from inductive assumption that $\mathcal{M}^{\mathsf{Real}}$ and $\mathcal{M}^{\mathsf{Ideal}}$ are computationally indistinguishable.

Step Type 3: Corruptions. We now handle corruption calls. Note that until the backup is uncompromised, that is, $\mathbb{A}(\mathsf{Corr} \cap \mathbb{S}) = 0$, the joint distribution between the two worlds are computationally indistinguishable, as we can directly use the same argument as in case 1 and 2 to argue the indistinguishability. Once the threshold is crossed, the argument doesn't hold anymore as we use the security properties of the sharing scheme, which won't hold above threshold. Therefore, now we need to show that after the backup is compromised, $\mathcal{A}$ still can't detect if it's in the ideal or real world.

To show this, we need to show that the joint distribution of the original backup post, along with the programmed oracle points, are indistinguishable in both worlds. This can be seen by our equivocation strategy: Sim makes sure that even though it sampled s_is randomly (unlike the real world), once it learns (m, pwd), it sets m_i's in a way to such that the distribution of the backup post is identical to it's distribution in the real world. Moreover, since we program the oracle with values $s_i - m_i$, where m_i is individually random, the programmed random oracle remains random. Finally, we notice that since we are equivocating *before* the adversary completely compromises the backup, the probability that simulation fails is negligible, as it needs to guess a missing k_i value to do so; guessing that happens with a negligible probability.

□